

What the meeting changed

Two people described the same next step in different words: connect through IMAP instead of the API, and build the mailbox before the assistant. This is what was asked, what was built, and the three bugs that turned up while building it.

THE ASK

What was actually said

SIR

"You have done the POC using the API. We need to do the POC using IMAP too... complete the implementation by noon tomorrow, for Google only for now."

*"It's not like we are not going to give the view of the mailbox to the user. We are definitely going to give it... **first we build the basic thing, like the mailbox, and then on top of that the agentic experience.**"*

UMAR

*"Sirf connection IMAP ke sath karna hai — baaki har cheez, aapka model, features, **sare same as it is kaam karenge.**"*

*"Sari mails ek jagah aa jayengi — un mein **messages include nahi honge, sirf headers...** sirf **last 30 days** ke message layenge... un mein se bhi **sirf important...** jab user header pe click karega wo **do second ke andar** fetch kar lega."*

They are describing one thing, not two

Sir asked for the door to change and the mailbox to come first. Umar described exactly what that mailbox holds. Neither is asking for a mail client — his own UC-27 rules that out: *"Out of scope: becoming a mail client... every company that built its own mail client here has died or been sold."*

CHANGE ONE

A second door into the same mailbox

The Gmail API and IMAP both reach the same mail. What differs is what Google charges for the privilege.

BEFORE — GMAIL API ONLY

- Google sign-in screen
- **Paid security audit every year**
- Weeks to months for approval
- 100 users until it clears
- Google pushes new mail to us



AFTER — EITHER DOOR, ONE FLAG

- ✓ App password — 16 letters
- ✓ **No audit. No fee.**
- ✓ No approval. Works today.
- ✓ No user cap
- ✓ We have to ask for new mail

Why the fee exists at all — it is a billing rule, not a technical one

Google's condition is: *use our sign-in screen to read somebody's Gmail, and you pay for an outside security assessment every year.* An app password never touches that screen, so the rule never applies. Same mail, same features, no gatekeeper.

the same command, both doors, same email:

```
python agent.py 2 --api
```

```
1 CONNECTOR      gmail · push=True labels=True
2 PARSE & STRIP   156,130 raw → 5,399 chars
9 NEEDS A REPLY? 0.14 — no
```

```
python agent.py 2 --imap
```

```
1 CONNECTOR      imap-gmail · push=False labels=True
2 PARSE & STRIP   156,130 raw → 5,399 chars
9 NEEDS A REPLY? 0.14 — no
```

Byte-identical stripping. Identical decision. Only the connector line differs.

Umar's "*sare same as it is kaam karenge*", demonstrated. One new file — `connectors/imap.py` — and nothing above it changed.

THE RESEARCH QUESTION SIR SET

Can IMAP give us only the Primary tab?

Yes. Verified on our own mailbox.

Gmail's IMAP server advertises X-GM-EXT-1 — Google's own extensions. One of them, X-GM-RAW, runs **the entire Gmail search box over IMAP.**

category:primary	1,747	
category:social	2,834	
category:promotions	1,260	
category:updates	3,554	
category:forums	0	
category:primary newer_than:30d	42	← exactly the slice Umar described

THE COMPARISON DOCUMENT SAID	ACTUALLY, ON GMAIL
Category tabs — TEST, unverified over IMAP	Works. 1,747 primary.
IMAP has no labels	Gmail does — X-GM-LABELS
IMAP has no threads	Gmail does — X-GM-THRID
IMAP cannot push	Correct. Still true.

But only on Gmail — and that is why it is a flag, not an assumption

Yahoo, Zoho and iCloud have no categories at all. So the connector **asks the server at connect time** rather than guessing from the hostname. Gmail-over-IMAP reports `supports_categories: yes`; Yahoo-over-IMAP reports no. The pipeline reads the flag and adapts.

CHANGE TWO

The mailbox — three tiers, exactly as specified

Umar described what it holds. This is that, built and measured on the real account.

EVERYTHING EVER	Headers only Sender, subject, date, which account. No message text at all.	7,391
LAST 30 DAYS	Headers + the text — but only what is worth reading Not sensitive, not machine-generated, not from a sender you never open. 118 skipped, 11 kept.	11 • 0.01 MB
OLDER	Headers. Text fetched when you click it. Gmail already holds your mail. A second copy would only save us a one-second fetch.	0.7–0.8s

```
python mailbox.py --open 25
```

Delivery Status Notification (Failure)

from Mail Delivery Subsystem · 2026-09-05T03:50

**** Address not found ****

Your message wasn't delivered to y1.waneringsoul@gmail.com

fetches live in 0.7s — never stored

Umar said two seconds. It is under one.

And the retention is not a promise on a page — `trim_bodies()` runs every time the view opens, so the 30-day window cannot quietly widen through neglect. The rule is code, and it executes whether or not anyone remembers it.

FOUND WHILE BUILDING

Three bugs, and they were all the same shape

None of them crashed. All three produced output that looked entirely reasonable.

1 Dates were stored as text, so nothing was in date order

14	Facebook	Wed, 9 Jun 2021
17	LinkedIn	Wed, 9 Jul 2025
22	AliExpress	Wed, 9 Aug 2023
27	Rashid	Wed, 9 Apr 2025

Every row starts "Wed, 9". Sorting that column sorted **alphabetically**, so 2021 sat beside 2025. Every "newest first" query in the project — the mailbox, the daily brief, the sorter — was returning an arbitrary mix and calling it recent.

Found because --warm looked at "the newest 400" and pulled zero. It was looking at 2021 mail.

Fixed: a real sortable date column, all 7,391 rows backfilled, every query switched.

2 Account-security mail was being read and cached

```
cached Your Instagram password has been changed
cached New Steam Account Email Verification
read Security alert from Google
read Your Google Account was recovered from Google
```

The patterns caught *"reset your password"* but not *"your password **has been changed**"*. **The request was blocked; the confirmation was not** — and the confirmation is the one that proves an account was taken over. That mail carries device names, locations and IP addresses.

Fixed: the gate now matches what security mail *says* rather than who sends it, so it works on providers we have never heard of. Also catches accounts.*, security.* and id.* subdomains.

3 A half-filled envelope made bounces look like human mail

The 30-day fetch built its message object without the Auto-Submitted field — the very field stage 4 reads to spot machine mail. So every message looked human-sent, and **seven "Delivery Status Notification (Failure)" bounces were confidently judged "worth keeping"** and their text stored.

```
21 pulled first attempt
-3 Instagram password, Steam verification — security mail
-7 bounce notifications
——
11 pulled real mail only
```

A partial object does not fail loudly. It produces a confident wrong answer, which is the harder kind to notice.

ALSO DONE

Corrections, widened

UC-29 asked for corrections that apply "to one sender, one category, or one kind of message". Only the first existed.

WHAT THE OWNER WANTS TO SAY	BEFORE	NOW
"never anything from @linkedin.com" — the whole company	no	scope = domain
"just this one message, not the sender"	silently wrong	pins to the Message-ID
"wrong label — that is an invoice, not an order"	no	yes
"this sender IS sensitive — never open them"	no	yes
"you warned me too late — warn me earlier"	no	--timing

One of those was worse than missing

"Only this one message" stored the *sender's address* regardless of scope — so the narrowest instruction the Owner can give was being applied as **"this sender, forever"**. A correction that spreads wider than asked is worse than one that fails, because nobody notices it happening.

And what the domain scope buys, measured

```
correct jobalerts-noreply@linkedin.com, scope = domain
```

```
jobalerts-noreply@linkedin.com → waiting
```

```
invitations@linkedin.com      → waiting ← never corrected
```

```
someone@gmail.com             → no correction
```

One correction covered **1,155 emails** from a LinkedIn address that was never touched.

WHERE IT STANDS

Asked, and done

ASKED BY	WHAT	STATE
Sir	POC on IMAP too, Google only for now	done — --imap on every command
Sir	Explore the differences	done — capabilities read at connect time
Sir	Find the question, bring the answer	done — Primary tab, 1,747
Sir	Build the mailbox first	done — mailbox.py
Umar	Only the connection changes	proven — identical output both doors
Umar	Headers for everything	7,391
Umar	Bodies for 30 days, important only	11, 0.01 MB
Umar	Click a header → ~2 seconds	0.7–0.8s

Not done, and not asked for

Outlook and JMAP connectors. Sir said Google only for now. Each is one file against an interface now proven by two implementations.

Push instead of polling. Needs a public address; a laptop has none. Becomes mandatory above roughly 150,000 users.

A real screen. Everything is still terminal text. Sir's order was mailbox first, then the agentic experience — the mailbox exists, as commands rather than a UI.

And one decision still open, for Umar

His own **BR-103**: *"Searching inside message text requires holding the words of every message... this is an open decision and must be settled before the storage design is fixed."*

The 30-day cache **partly answers it** — we now store text, but bounded and auto-deleted. That is a defensible middle position: *"we hold 30 days of your important mail"* is a sentence a customer can weigh. Worth confirming that bounded storage is what he intended, because it changes what UC-32 can claim.